

Getting Background: Tile Engine, Dispatcher, and Your Task

A complete guide — from zero to expert

Part 1: Start From Absolute Zero

What does a GPU actually do?

A GPU (Graphics Processing Unit) is a chip originally designed to render video game graphics. To draw a frame, it needs to apply the same math operation to millions of pixels simultaneously. Engineers discovered that same ability — doing the same calculation on millions of numbers in parallel — is exactly what AI and deep learning need.

When you train a neural network, you are mostly multiplying large tables of numbers together, millions of times. GPUs are extraordinarily good at this. A modern AMD GPU like the MI300X can do roughly 1,300 trillion math operations per second (1.3 PFLOP/s). That number only happens if the GPU is programmed correctly.

What is a matrix?

A matrix is just a table of numbers arranged in rows and columns:

Matrix A (3 rows, 3 columns):

1	2	3
4	5	6
7	8	9

Matrix B (3 rows, 3 columns):

7	8	9
1	2	3
4	5	6

Matrix multiplication (GEMM — General Matrix Multiply) takes two matrices and produces a third. Every single cell of the result is the dot product of a row from A and a column from B. If A is 1024 rows × 1024 columns and B is 1024 × 1024, computing $C = A \times B$ requires roughly 2 billion multiply-add operations. That is why GPUs are essential — a CPU would take seconds; a GPU takes milliseconds.

What is a kernel?

A **kernel** is a small program that runs directly on the GPU hardware. It is not a Python script or a normal program — it is compiled machine code that runs on thousands of GPU cores simultaneously.

Think of a kernel like a **recipe card** for one specific cooking task:

- It says exactly which ingredients to pick up (load from GPU memory)
- Exactly how to combine them (multiply, add, accumulate)
- Exactly where to put the result (store back to GPU memory)
- How many cooks work simultaneously (thread count, block size)

A kernel must be written very precisely for the specific GPU hardware. The wrong memory access pattern, the wrong tile size, the wrong instruction sequence — and the GPU either crashes or runs 10× slower than it should.

Why are there thousands of kernels?

You might think: one kernel for matrix multiply, done. But in practice, every variation needs its own optimized kernel:

- **Data type:** fp16 (16-bit float), bf16 (brain float), fp8, int8, fp32 — each has different memory width, different hardware instructions, different valid tile sizes - **Matrix layout:** Row-major (rcr), column-major (ccr), and combinations — the physical memory arrangement changes how data is loaded - **Tile shape:** How big a chunk each group of GPU cores works on at once — 256×128×32, 128×128×64, etc. Different shapes suit different problem sizes - **Pipeline:** The internal execution strategy — compv3, compv4, preshufflev2 — controlling how data flows through registers and shared memory - **GPU architecture:** MI300X (gfx942), MI250X (gfx90a), RX 7900 (gfx1100) — each GPU has different instruction sets and memory hierarchies - **Padding:** Whether matrix dimensions happen to be multiples of the tile size (clean) or not (requires edge-case handling)

Multiply all these options together: 6 data types × 4 layouts × 20 tile shapes × 5 pipelines × 7 GPU targets × 2 padding modes = **16,800 kernels**. Nobody writes 16,800 kernels by hand.

What is a code generator (codegen)?

A **code generator** is a Python script that writes kernel code automatically. You give it a description of what you want (data type, layout, tile shape, etc.) and it writes the C++ kernel code for you.

Think of it like a **form letter generator** at a law office. The lawyer defines a template; the assistant fills in the client name, date, and specific terms to produce a customized letter in seconds. The codegen does the same: it has templates for each kernel variant, and it fills in the specific configuration parameters to produce ready-to-compile C++ code.

The output is a `.hpp` header file — a chunk of C++ code that, when compiled, becomes a fast GPU kernel.

Part 2: Tile Engine — The Old Way

The core idea

Tile Engine was AMD's first serious attempt at organizing this kernel generation problem. The approach: **for every configuration combination, generate and compile a dedicated binary executable**.

Here is the pipeline:

Step 1: You write a JSON config file describing what you want

```
{
  "datatype": "fp16",
  "layout": "rcr",
  "tile_config": { "tile_m": 256, "tile_n": 128, "tile_k": 32 },
  "trait_config": { "pipeline": "compv3", "scheduler": "intrawave" }
}
```

Step 2: A Python codegen script reads the config and writes C++ kernel code

```
unified_gemm_codegen.py --config fp16_rcr.json → gemm_fp16_rcr_compv3_...hpp
```

Step 3: The C++ compiler compiles that header into a standalone executable

```
hipcc gemm_fp16_rcr_compv3_...hpp → benchmark_gemm_universal_fp16_rcr_compv3_
...
```

Step 4: To run the kernel, you call that binary directly
./benchmark_gemm_universal_fp16_rcr_compv3_intrawave_256x128x32 -m=1024 -n=10
24 -k=1024
→ Prints: TFLOP/s = 84.7

What Tile Engine looks like from the outside

Every configuration has its own named binary. If you list the TE build directory, you see something like:

```
benchmark_gemm_universal_fp16_rcr_compv3_default_intrawave_False_False_False_False_256
x128x32_4x1x1_32x32x16
benchmark_gemm_universal_fp16_rcr_compv3_default_intrawave_False_False_False_False_128
x128x32_2x2x1_32x32x16
benchmark_gemm_universal_fp16_rcr_compv4_default_intrawave_False_False_False_False_256
x256x32_4x2x1_32x32x16
benchmark_gemm_universal_bf16_rcr_compv3_default_intrawave_False_False_False_False_256
x128x32_4x1x1_32x32x16
... (hundreds more)
```

Each binary encodes its full configuration in its name. CI scripts call them by name and capture the TFLOP/s output.

The problems with Tile Engine

Problem 1 — Build explosion. 500 configurations = 500 separate compilation jobs. Each job takes time. A full CI build can take hours. Adding a new GPU architecture means recompiling everything.

Problem 2 — Hard to scale. As the number of valid tile configurations grows, so does the number of binaries linearly. There is no sharing of compilation output between similar kernels.

Problem 3 — No runtime flexibility. The kernel is locked in at compile time. To pick a different kernel for a different problem size, you call a different binary. There is no central intelligence to choose the best kernel for a given situation.

Problem 4 — Duplication. The kernel logic inside each binary is nearly identical — only a few template parameters differ. Compiling 500 nearly-identical programs is wasteful.

The restaurant analogy: Tile Engine is a restaurant where every dish has its own dedicated kitchen, its own dedicated chef, and its own dedicated dining room. You want spaghetti? Kitchen 47. You want pizza? Kitchen 83. 500 dishes = 500 separate kitchens. Expensive, hard to maintain, impossible to scale.

Part 3: The Dispatcher — The New Way

The core idea

The Dispatcher solves the same problem differently: **compile all kernels into one shared library, register them in a central catalog, and dispatch the right one at runtime.**

Here is the pipeline:

Step 1: CMake build runs the SAME codegen script, but generates ALL kernels

unified_gemm_codegen.py → generates hundreds of `gemm_*.hpp` files

Step 2: ALL those headers are compiled together into ONE shared library
`hipcc [all kernel headers] → libdispatcher_gemm_lib.so`

Step 3: Inside the `.so`, every kernel registers itself in a central Registry
The registry is like a phone book: kernel name → kernel object
Registry:
 `"fp16_rcr_compv3_...256x128x32..." → KernelObject_A`
 `"fp16_rcr_compv4_...256x128x32..." → KernelObject_B`
 `"bf16_rcr_compv3_...256x128x32..." → KernelObject_C`
 ... (all kernels in one place)

Step 4: A Python interface loads the `.so` and dispatches kernels on demand
from `ctypes_utils import DispatcherLib`
`lib = DispatcherLib.load()` # load the `.so`
`lib.run_gemm(A, B, C, M=1024, N=1024, K=1024, dtype="fp16", ...)`
→ Dispatcher looks up the registry → picks best kernel → runs it
→ Returns: `time_ms = 11.8, tflops = 84.7`

What the Dispatcher looks like from the outside

From Python, it is just a function call:

```
from ctypes_utils import DispatcherLib

lib = DispatcherLib.load("/path/to/libdispatcher_gemm_lib.so")

# Run a GEMM – dispatcher picks the right kernel automatically
result = lib.run_gemm(
    A_gpu, B_gpu, C_gpu,
    M=1024, N=1024, K=1024,
    dtype="fp16"
)
print(f"Time: {result.time_ms:.2f} ms, TFLOP/s: {result.tflops:.1f}")
```

No binary names. No configuration strings. No manual kernel selection. Just: give me the matrices, tell me the size and dtype, get the result.

How the Dispatcher chooses a kernel

Inside the Dispatcher, each kernel is stored with a unique identifier — a string that encodes all its configuration parameters:

```
"fp16_rcr_compv3_default_intrawave_False_False_False_False_256x128x32_4x1x1_32x32x16"
  ^^^^^  ^^^  ^^^^^  ^^^^^^^^^  ^^^^^^^^^  ^^^^^  ^^^^^  ^^^^^  ^^^^^^^^^  ^^^^^^^^^^^^^^^^^^^^^
dtype layout pipe epilogue scheduler  padM   padN   padK  persist  tile × warp × wt
```

This string is the kernel's registry key. The C++ `kernelKey` struct holds all the parameters, and its `encode_identifier()` method produces this string. The registry maps string → kernel object. Looking up a kernel is just a dictionary lookup.

The problems the Dispatcher solves

Solution 1 — One binary instead of hundreds. All kernels share one `.so` file. Build once, use any kernel.

Solution 2 — Runtime flexibility. The Dispatcher can choose the best kernel for a given problem size at runtime, not at compile time. A heuristic or ML model can guide selection.

Solution 3 — Python-first interface. [DispatcherLib](#) gives a clean Python API. No bash scripts calling binary names.

Solution 4 — Single source of truth. One registry. One place to add, remove, or update kernels. No duplication.

The restaurant analogy: The Dispatcher is a modern restaurant with ONE professional kitchen and ONE full menu. When you order, the kitchen checks its menu (the registry), selects the right technique for that dish, and cooks it. 500 dishes = 1 kitchen, 500 entries in the menu. Clean, efficient, scalable.

Part 4: The C++ Internals (For the Expert Layer)

You do not need this to understand the task. But if you want to be an expert, here is what is actually happening inside the Dispatcher.

KernelKey — The identity card of every kernel

```
struct KernelKey {
    struct Signature {          // WHAT is being computed
        DataType dtype_a, dtype_b, dtype_c, dtype_acc;
        LayoutTag layout_a, layout_b, layout_c;
        uint8_t split_k;
        std::string elementwise_op; // "PassThrough", "Relu", etc.
        bool structured_sparsity;
    } signature;

    struct Algorithm {          // HOW it is computed
        struct TileShape { uint16_t m, n, k; } tile_shape;    // 256, 128, 32
        struct WaveShape { uint8_t m, n, k; } wave_shape;    // 4, 1, 1
        struct WarpTile { uint8_t m, n, k; } warp_tile;      // 32, 32, 16
        Pipeline pipeline;    // CompV3, CompV4, PreShuffleV2...
        Scheduler scheduler;   // Auto, Intrawave, Interwave
        Epilogue epilogue;    // Default, CShuffle...
        bool pad_m, pad_n, pad_k;
        bool double_buffer, persistent, preshuffle;
    } algorithm;

    std::string encode_identifier() const; // produces the registry key string
};
```

Registry — The phone book

```
class Registry {
    // Thread-safe map: identifier string → KernelInstance shared_ptr
    std::map<std::string, KernelInstancePtr> kernels_;
    std::mutex mutex_;

public:
    bool register_kernel(KernelInstancePtr kernel);
    KernelInstancePtr lookup(const std::string& identifier) const;
    KernelInstancePtr lookup(const KernelKey& key) const; // calls encode_identifier
```

```
std::vector<KernelInstancePtr> get_all() const;
};
```

KernelInstance — The abstract kernel interface

```
class KernelInstance {
public:
    virtual const KernelKey& get_key() const = 0;
    virtual bool supports(const Problem& problem) const = 0;
    virtual float run(const void* a, const void* b, void* c,
                     const Problem& problem, void* stream) const = 0;
};
```

Every generated kernel is a concrete class that inherits this interface. The codegen writes these concrete classes automatically.

Dispatcher — The orchestrator

```
class Dispatcher {
    Registry* registry_;
    HeuristicFunction heuristic_; // optional ML-based kernel picker

public:
    // Automatic selection: looks up registry, runs best kernel
    float run(const void* a, const void* b, void* c,
             const Problem& problem, void* stream) const;

    // Explicit selection: you name the kernel by identifier
    float run_explicit(const std::string& kernel_id, ...);

    // Selection only: returns the kernel without running it
    KernelInstancePtr select_kernel(const Problem& problem) const;
};
```

The ctypes bridge — How Python talks to C++

The `.so` file exposes a C API (not C++ — C has a simpler calling convention that Python can call directly):

```
extern "C" {
    int gemm_dispatcher_initialize(const char* arch);
    float gemm_dispatcher_run(float* a, float* b, float* c,
                             int M, int N, int K, const char* dtype);
    void gemm_dispatcher_cleanup();
}
```

Python's `ctypes` library loads the `.so` and calls these C functions as if they were Python functions. `ctypes_utils.py` wraps this in a clean Python class so callers never see the low-level details.

Part 5: The Warp/Wave Naming Trap (A Critical Detail)

This is the most dangerous source of silent bugs in this codebase. It is worth understanding deeply.

The GPU hardware has a hierarchy of execution units:

GPU

■■■ Compute Units (CUs)

■■■ Wavefronts / Warps ← a group of 64 threads that execute together

■■■ Individual threads

A "warp" and a "wavefront" are the same concept — AMD hardware calls it a wavefront (64 threads), NVIDIA hardware calls it a warp (32 threads). The CK codebase uses both terms, sometimes interchangeably.

In a GEMM kernel, the computation is tiled at multiple levels:

Block tile (what one GPU compute block handles): e.g. 256×128

■■■ Wave tiles (what one wavefront handles): e.g. 64×128 (4 waves per block in M)

■■■ Warp tile / MFMA shape (what one warp's matrix instruction computes):
e.g. 32×32 per instruction

The trap: Tile Engine and the Dispatcher use the words "warp" and "wave" inconsistently:

Parameter	Tile Engine meaning	Dispatcher meaning
<code>warp_m / warp_n / warp_k</code>	How many wavefronts tile the block (wave COUNT)	<code>wave_shape</code> — same concept
<code>warp_tile_m/n/k</code>	The MFMA instruction shape per wavefront	<code>warp_tile</code> — same concept

If you swap `warp_m` (wave count = 4) with `warp_tile_m` (MFMA shape = 32), you get a config that looks valid but produces wrong answers — and there is no error message. The kernel launches, runs, and silently computes garbage. This is documented in `te_to_dispatcher.py:_Tile` with a big warning comment.

Part 6: What the Project Description Is Actually Asking

Now that you understand both systems, re-read the Confluence page goal:

> "Replace Tile Engine with the dispatcher by achieving feature parity using the > dispatcher's design, then routing all Tile Engine functionality through it."

Let us break this into concrete sentences:

Sentence 1: "Replace Tile Engine with the dispatcher"

Delete the Tile Engine system. It will no longer exist. Every place that currently calls a TE binary or uses TE JSON configs must be updated to use the Dispatcher.

Sentence 2: "achieving feature parity using the dispatcher's design"

Before you delete anything, prove the Dispatcher can do everything TE could do. But — and this is critical — prove it **using the Dispatcher's own interface**, not by translating TE configs and running them through a custom harness.

Parity means: - Same math answer (within tolerance) - Same speed (within 2%) - Every dtype/layout/pipeline combination that TE covered

Sentence 3: "routing all Tile Engine functionality through it"

Every caller that used TE must now go through `DispatcherLib`. The Dispatcher is the only path. TE code is removed.

Sentence 4: "Do NOT make the dispatcher look like Tile Engine"

Do not build a translator that takes TE JSON and converts it to Dispatcher format. That would be making the Dispatcher conform to TE's vocabulary. Instead, callers must learn the Dispatcher's vocabulary and call it directly. TE conforms to the Dispatcher, not the other way around.

Part 7: Why the Current PR Missed the Mark

The current PR built a translation layer:

TE JSON → `te_to_dispatcher.py` → config dict → `drive_codegen.py` → `harness.cpp` → compare

There are three specific problems:

Problem 1 — The translator preserves TE as the source of truth

`te_to_dispatcher.py` takes TE JSON as input. This means: to use the Dispatcher, you still need to start with a TE JSON config. TE's vocabulary is still driving things. Vidya said explicitly: do not do this. The Dispatcher should be driven by its own API, not converted-TE configs.

Problem 2 — `drive_codegen.py` treats the Dispatcher's own tool as a TE artifact

`unified_gemm_codegen.py` lives in `dispatcher/codegen/`. It is the Dispatcher's tool. CMake already calls it during the normal Dispatcher build. The PR's `drive_codegen.py` invoked it as an external subprocess to generate a one-off header for a custom harness — bypassing the actual Dispatcher build system and treating the Dispatcher's internal tool as if it were a TE utility.

Problem 3 — `harness.cpp` is not the Dispatcher's Python interface

The Dispatcher already has `ctypes_utils.py::DispatcherLib` — the actual Python runtime interface that will replace TE in production. The PR built a custom `harness.cpp` single-kernel C++ runner instead. Proving the custom harness gives correct answers does not prove `DispatcherLib` gives correct answers. These are different things. Only `DispatcherLib` is the real replacement.

What the PR proved vs what it should have proved

	Current PR	What was needed
Proved	Custom translator + custom harness agree with TE	DispatcherLib agrees with TE
Tested	te_to_dispatcher.py translation logic	The actual production interface
End state	Two systems running + a parity harness	TE deleted, DispatcherLib is the only path

Part 8: The Right Approach — Step by Step

Here is what a great delivery looks like:

Step 1 — *FIND the TE call sites*

Before writing a single line of code, find every place that currently calls TE:

```
# Search for benchmark binary calls in CI scripts
grep -r "benchmark_gemm_universal" /path/to/ci_scripts/
grep -r "benchmark_gemm_universal" /path/to/repo/

# Search for TE JSON config usage
grep -r "tile_config" /path/to/repo/ --include="*.json"
grep -r "trait_config" /path/to/repo/ --include="*.json"
```

This gives you the exact list of things to replace. Without this list, you are guessing at scope.

Step 2 — *INVENTORY what kernels are actually used*

For each call site, note: what dtype, layout, pipeline? What problem sizes (M, N, K)? This becomes your parity test matrix. If CI only ever runs fp16 rcr compv3 on 1024×1024×1024, that is your parity scope — not all 16,000 possible combinations.

Step 3 — *VERIFY the Dispatcher already has those kernels*

Build the Dispatcher normally:

```
cd dispatcher && cmake -B build && cmake --build build
```

Then check what kernels are registered:

```
from ctypes_utils import DispatcherLib
lib = DispatcherLib.load()
for kernel in lib.list_kernels():
    print(kernel.identifier)
# → see if fp16_rcr_compv3...256x128x32... is in the list
```

If the kernel is already there: move to Step 4. If it is missing: this is a gap in the Dispatcher's build config that needs to be fixed before you can proceed.

Step 4 — WRITE parity tests through DispatcherLib

For each (kernel, problem size) pair from Step 2:

```
# Run through DispatcherLib
lib = DispatcherLib.load()
result_disp = lib.run_gemm(A, B, C, M=1024, N=1024, K=1024, dtype="fp16")

# Run through TE binary
result_te = subprocess.run(
    ["benchmark_gemm_universal_fp16_rcr_compv3_...", "-m=1024", "-n=1024", "-k=1024"]
    ,
    capture_output=True, text=True
)

# Assert parity
assert abs(result_disp.tflops - result_te.tflops) / result_te.tflops < 0.02
assert numerical_match(result_disp.C, result_te.C, atol=1e-3, rtol=1e-2)
```

This is the proof certificate that makes deletion safe.

Step 5 — WRITE the thin Python frontend

Replace the TE binary interface with a Python function:

```
# Before (TE way - calls a binary by name):
# ./benchmark_gemm_universal_fp16_rcr_compv3_... -m=1024 -n=1024 -k=1024

# After (Dispatcher way - calls Python interface):
def run_gemm_benchmark(dtype, layout, M, N, K, pipeline=None):
    lib = DispatcherLib.load()
    result = lib.run_gemm(A, B, C, M=M, N=N, K=K, dtype=dtype)
    return result.tflops
```

This is thin — it does not know anything about kernel internals. It just passes the order to [DispatcherLib](#) and returns the result.

Step 6 — UPDATE the call sites

Change each CI script or benchmark script found in Step 1 to call the thin frontend instead of the TE binary.

Step 7 — DELETE Tile Engine

Remove: - The `benchmark_gemm_universal_*` build targets - The TE JSON config files that drove codegen for TE binaries - Any TE-specific build scripts

Keep: - `dispatcher/codegen/unified_gemm_codegen.py` — this is the Dispatcher's tool - `dispatcher/python/ctypes_utils.py` — this is the replacement interface - `dispatcher/` everything else — it all belongs to the Dispatcher

Step 8 — DOCUMENT

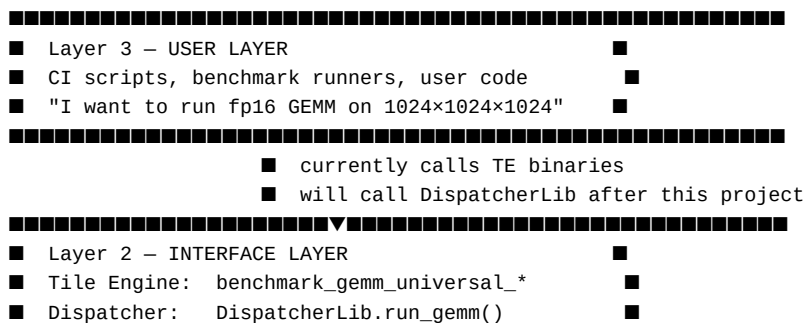
Write a short document: - What was deleted - What replaced it - How to run the new thin frontend - Where the parity evidence lives

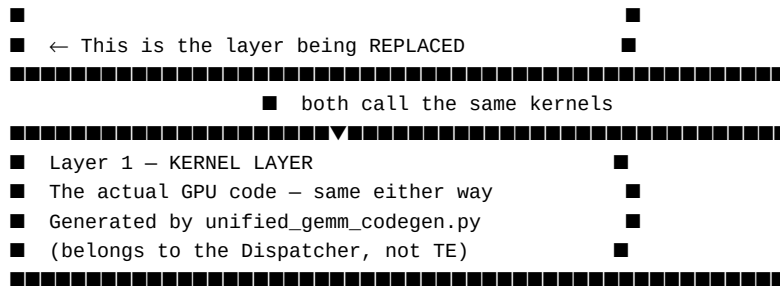
Part 9: The Vocabulary You Need to Know

Term	Plain meaning
Kernel	A GPU program — the actual code that runs on hardware
GEMM	Matrix multiply — the core computation
Codegen	A Python script that automatically writes kernel code
Tile	A chunk of the matrix one group of GPU cores handles at once
Pipeline	The internal data-flow strategy inside a kernel (compv3, compv4...)
Scheduler	Controls how waves of threads are ordered (intrawave, interwave)
Registry	The phone book — maps kernel name → kernel object
KernelKey	The identity card — encodes all parameters of one kernel
encode_identifier()	Turns a KernelKey into the registry lookup string
DispatcherLib	The Python class that loads the .so and dispatches kernels
Parity	Both systems produce the same answer at the same speed
Thin frontend	A small Python wrapper that calls DispatcherLib — the TE replacement
Warp	A group of GPU threads executing together (64 on AMD)
Wave	Same as warp on AMD hardware
MFMA	Matrix Fused Multiply-Add — the GPU's hardware matrix instruction
warp_tile	The shape one warp computes with one MFMA instruction
wave_shape	How many waves tile the block in each dimension

Part 10: The One Mental Model That Makes Everything Click

Think of it as a three-layer cake:





Layer 1 (the actual kernels) does not change. Layer 3 (the callers) changes minimally. Layer 2 is the thing being replaced — TE's binary interface goes away, [DispatcherLib](#) takes its place.

Your job is to swap out Layer 2, prove it produces the same results, and remove the old version.

Summary: What This Project Is, In One Paragraph

AMD has two systems for running GPU matrix-multiply kernels — Tile Engine (old, per-binary, hard to scale) and Dispatcher (new, one library, Python interface). Both do the same thing. The project's goal is to prove the Dispatcher can replace Tile Engine completely, then delete Tile Engine. "Proof" means: running the same problem sizes through the Dispatcher's Python interface ([DispatcherLib](#)) and getting the same math answers at the same speed as the Tile Engine binaries. The proof must go through the real production interface — not a custom harness, not a translator. Once the proof is in hand, every call site that invoked TE binaries is updated to call [DispatcherLib](#) instead, and the TE code is deleted. The Dispatcher becomes the single system. Done.